ADR-002 · DÉCISION D'ARCHITECTURE

Choix de la base de données : PostgreSQL ou MongoDB

Étude des avantages techniques d'une base relationnelle et d'une base documentaire pour le backend de la table de jeu de rôle virtuelle (sujet EpiQuest, Programme Grande École, 5e année)

Statut	**Accepté**	**Date**	28 octobre 2026
Auteur	Quentin, responsable backend	**Périmètre**	Persistance du serveur : comptes, campagnes, règles, cartes, combats
Décision	**PostgreSQL en modèle hybride** : tables relationnelles + colonnes JSONB	**Lié à**	ADR-001 (C# / .NET 10, EF Core)

01 Résumé de la décision

> Le serveur utilise **PostgreSQL** en **modèle hybride** : des tables relationnelles pour le cœur (comptes, campagnes, personnages, cartes, combats) et des colonnes **JSONB** pour les données semi-structurées (définitions du SRD, format des effets, choix de création, géométrie des cartes).

La question n'est pas vraiment « relationnel contre document » : PostgreSQL avec JSONB couvre une bonne partie de ce que MongoDB apporte. Le cœur du sujet (combats atomiques et annulables, permissions par campagne, import versionné du SRD) repose sur l'intégrité et les transactions, qui sont les points forts de PostgreSQL. MongoDB garde l'avantage pour stocker des agrégats autonomes, mais ce n'est pas le profil de ce projet.

02 Les données du projet

Les données du sujet se répartissent en deux familles :

- **Des données fortement relationnelles** : utilisateurs, campagnes, appartenance avec un rôle (MJ dans l'une, joueur dans l'autre), personnages rattachés à une campagne, cartes, pions, rencontres, combats, jets. Les permissions vérifiées côté serveur traversent toutes ces relations.
- **Des données semi-structurées** : définitions du SRD (sorts, monstres, capacités) et format des effets, de structure hétérogène et polymorphe. S'y ajoutent les choix de création d'un personnage et la géométrie des cartes (murs, portes).

Exigence du sujet	Réf.	Ce qu'elle demande à la base
Résultat d'une action modifiable, annulable et historisé	02-H, A1	Écrire plusieurs entités de façon atomique ; journal ordonné des actions.
Permissions vérifiées côté serveur (MJ, joueur, campagne)	03-B	Requêtes transverses fiables entre utilisateurs, campagnes et personnages.
Import du SRD rejouable et versionné ; règles décrites comme des données	02-C	Stocker des documents hétérogènes, rattachés à une version d'import.
Bestiaire : recherche et filtres (type, facteur de puissance, taille)	02-G	Index sur des attributs et recherche textuelle tolérante.

Exigence du sujet	Réf.	Ce qu'elle demande à la base
Installation en une commande ; nouveau contributeur opérationnel en 15 minutes	03-B	Démarrage local simple, sans étape d'initialisation manuelle.

03 Avantages techniques de PostgreSQL

1. **Transactions multi-lignes ACID, sans configuration.** C'est l'argument le plus fort pour ce sujet. Une boule de feu sur quatre cibles modifie les PV de quatre créatures, consomme un emplacement de sort, ajoute un état et une entrée de journal. Tout doit être appliqué ensemble ou pas du tout, et l'annulation aussi : un simple `BEGIN … COMMIT` suffit.
2. **Intégrité référentielle dans le moteur.** Une clé étrangère `character.campaign_id` avec la règle `ON DELETE` adaptée, une contrainte `UNIQUE (campaign_id, user_id)` sur l'appartenance, des `CHECK` sur les valeurs (PV ≥ 0, niveau entre 1 et 20). Si le code applicatif a un bug, la base refuse l'écriture au lieu de stocker un état incohérent.
3. **JSONB, le meilleur des deux mondes pour les règles.** Une table `srd_spell` combine des colonnes indexées (`name`, `level`, `school`) et une colonne `definition jsonb` pour le format d'effets polymorphe. Les index GIN, l'opérateur `@>` et `jsonb_path_query` interrogent l'intérieur des documents ; un `CHECK` peut imposer un champ `type` dans chaque effet. Npgsql mappe ces colonnes vers des types C# avec EF Core.
4. **Recherche dans le bestiaire.** Les filtres sont de simples index B-tree. `pg_trgm` gère la tolérance aux fautes (« gobelin » contre « goblin ») et `tsvector` avec `unaccent` le plein texte en français, sans service externe.
5. **Requêtes transverses avec jointures.** « Tous les personnages des campagnes où cet utilisateur est MJ, avec leurs PV actuels » s'écrit en une jointure optimisée par le planificateur. La vérification des permissions repose sans cesse sur ce type de requête.
6. **Versionnement de l'import SRD.** Une table `srd_version` est référencée par chaque entité importée. Une campagne peut être épinglée sur une version des règles, et le rejeu de l'import se fait dans une transaction : il réussit en entier ou ne laisse aucune trace.
7. **Outillage.** Migrations EF Core versionnées dans le dépôt, Testcontainers pour une base éphémère dans les tests, image Docker officielle qui démarre en une ligne de `docker-compose.yml`.

04 Avantages techniques de MongoDB

1. **Le modèle document correspond aux agrégats.** Une fiche de personnage, avec son inventaire, ses sorts préparés, ses choix de création et ses notes, se lit et s'écrit en une seule opération. De même pour une carte avec ses calques, ses murs, ses portes et ses pions : charger l'état complet d'une partie est très naturel.
2. **Atomicité par document sans transaction.** Si le combat est un seul document contenant tous les combattants, une action se résume à un `updateOne` avec `$inc`, `$set` et `$push`, atomique par nature. Le journal doit en revanche sortir dans une collection séparée, à cause de la limite de 16 Mo par document.
3. **Import du SRD sans friction.** Les structures hétérogènes d'Open5e ou de la 5e API s'insèrent telles quelles, et un nouveau champ ne demande aucune migration. Le driver C# gère nativement les discriminateurs polymorphes (`_t`, `[BsonKnownTypes]`), ce qui convient bien au format des effets.
4. **Change streams.** Ils permettent de s'abonner aux modifications d'une collection pour alimenter la synchronisation des vues. L'intérêt est limité ici : le serveur fait autorité et pousse déjà les changements via SignalR au moment où il les écrit.

5. **Évolution rapide du schéma.** Au début du projet, quand le format des effets change chaque semaine, ne pas écrire de migration fait gagner du temps.

05 Comparaison sur les besoins du sujet

PostgreSQL l'emporte sur l'atomicité, l'intégrité et l'installation ; MongoDB sur le stockage naturel des agrégats. Vert : avantage net, jaune : correct avec un effort, rouge : point faible.

Besoin du sujet	PostgreSQL	MongoDB
Action de combat multi-cibles, atomique et annulable	Transaction native	Atomique si tout est dans un seul document ; sinon transaction multi-documents, qui exige un *replica set*
Comptes, campagnes, rôles, permissions	Clés étrangères, jointures, contraintes	Intégrité à la charge du code, `$lookup` moins performant
Définitions de règles polymorphes	JSONB + colonnes indexées	Natif
Fiche de personnage, état d'une carte	JSONB pour les parties imbriquées	Natif, un document par agrégat
Recherche dans le bestiaire	`pg_trgm`, `tsvector`, sans service externe	Index texte basiques ; la recherche avancée passe par Atlas Search
Import rejouable et versionné	Transactionnel, référencé par clé étrangère	Simple à insérer, cohérence du rejeu à gérer
Installation en une commande	Une seule image	Le *replica set* mono-nœud doit être initialisé pour les transactions et les change streams
Écosystème .NET	EF Core + Npgsql, migrations	Driver officiel solide ; provider EF Core plus jeune et moins complet

> **Le piège du replica set.** Sans lui, ni les transactions multi-documents ni les change streams de MongoDB ne fonctionnent. Il faut donc un script d'initialisation dans le `docker-compose`, ce qui complique un peu le test du nouvel arrivant (15 minutes avec le seul README).

06 Décision : PostgreSQL en modèle hybride

La souplesse documentaire n'est utile que sur quelques zones du modèle, et JSONB les couvre. Chaque donnée est rangée selon sa nature :

Donnée	Stockage
Utilisateurs, campagnes, appartenances, personnages, cartes, rencontres, combats	Tables relationnelles
Définitions SRD (sorts, monstres, capacités, effets)	Colonnes clés indexées + `definition jsonb`
Choix de création, inventaire, notes du personnage	`jsonb` dans la table du personnage
Murs, portes, décor d'une carte	`jsonb` (liste de segments), chargé en mémoire pour la ligne de vue
Journal de combat	Table en ajout seul : `(combat_id, seq bigserial, type, payload jsonb)`

Illustration du modèle hybride sur deux tables :

```
CREATE TABLE srd_spell (
    id           uuid PRIMARY KEY,
    version_id   uuid NOT NULL REFERENCES srd_version(id),
    slug         text NOT NULL,
    name         text NOT NULL,
    level        smallint NOT NULL CHECK (level BETWEEN 0 AND 9),
    school       text NOT NULL,
    definition   jsonb NOT NULL CHECK (definition ? 'resolution'),
    UNIQUE (version_id, slug)
);
CREATE INDEX ON srd_spell USING gin (definition jsonb_path_ops);

CREATE TABLE combat_event (
    combat_id    uuid NOT NULL REFERENCES combat(id) ON DELETE CASCADE,
    seq          bigserial,
    type         text NOT NULL,          -- action, effet_mj, annulation...
    payload      jsonb NOT NULL,
    created_at   timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (combat_id, seq)
);
```

07 Conséquences

Positives	Négatives et mesures prises
Une action de combat et son annulation sont atomiques, quel que soit le nombre de cibles.	Les migrations demandent de la discipline. *Mesure :* migrations EF Core versionnées, appliquées automatiquement au démarrage en local, et vérifiées en CI.
La base garantit l'intégrité des campagnes, des rôles et des personnages, même en cas de bug applicatif.	Le contenu des colonnes JSONB est moins contraint que des colonnes typées. *Mesure :* validation par le JSON Schema du format des effets à l'import, et contraintes CHECK minimales.
Une seule image Docker, sans initialisation : le test du nouvel arrivant reste simple.	Une fiche de personnage se répartit entre colonnes et JSONB. *Mesure :* l'agrégat est chargé en une requête et reconstruit en C# par EF Core.
Le journal de combat est une table ordonnée : rejeu et annulation s'écrivent en SQL simple.	Le format des effets évolue vite au début. *Mesure :* le JSONB absorbe ces changements sans migration de colonnes ; seul l'import est rejoué.

08 Critères de réévaluation

La décision sera réexaminée si :

- le groupe choisit de modéliser tout le jeu en agrégats autonomes (un document par combat, un par carte), sans requêtes transverses ;
- le format des données de règles devient si instable que même le JSONB et les migrations freinent le travail ;
- les mesures de performance montrent que le chargement de l'état d'une partie devient un goulot d'étranglement, ce qui justifierait un cache en mémoire avant de changer de base.

Sources

- Sujet EpiQuest, Programme Grande École 5e année, Epitech Paris, 2026-2027 (blocs 01 à 05).
- ADR-001, Choix du langage et de la plateforme serveur, 28 octobre 2026.